Solute Redistribution in Solidification

Interactive Slider Applications and a Temperature-Coupled Microsegregation Model

Technical Report

MSE 670 — Solidification

Author	Prathamesh Deepak Ladhe
Course	MSE 670, Solidification
Deliverables	Browser app (`index.html`), MATLAB app (`fs_slider.m`), five standalone model functions, documentation
Live demo	https://1clan.github.io/mse670-solute-redistribution/
Repository	`fs_slider_project`
Date	November 13, 2026

Scope. This report documents the physical model, the numerical implementation, the browser front end and the MATLAB front end. It states every equation that is implemented, every default parameter with its provenance, every simplifying assumption, and the tests used to verify that the two front ends agree.

Contents

1 Introduction and scope **3**
1.1 Purpose . . . 3
1.2 Deliverables . . . 3
1.3 What is template and what is authored . . . 3
1.4 How to read this report . . . 4

2 Physical model and notation **4**
2.1 The one-dimensional solidification problem . . . 4
2.2 Equilibrium: the lever rule . . . 4
2.3 Scheil–Gulliver: no solid diffusion, mixed liquid . . . 5
2.4 No solid diffusion with limited liquid diffusion . . . 5
2.5 Clyne–Kurz: finite back diffusion . . . 6
2.6 Won–Thomas coarsening correction . . . 7
2.7 Temperature coupling . . . 7
2.8 Numerical edge cases . . . 9

3 The browser application **9**
3.1 Architecture . . . 9
3.2 State and update flow . . . 10
3.3 Controls and override semantics . . . 10
3.4 Charts . . . 10
3.5 Animation . . . 11
3.6 Responsive layout and accessibility . . . 11
3.7 Performance . . . 11

4 The MATLAB application **11**
4.1 Requirements and how to run . . . 11
4.2 Architecture: one function, nested callbacks . . . 12
4.3 The UI template part . . . 12
4.4 Callbacks and the update cycle . . . 12
4.5 Animation . . . 13
4.6 Plotting and panels . . . 13
4.7 File-by-file guide . . . 13
4.7.1 `fs_slider.m` — the app . . . 13
4.7.2 `models/equilibrium_model.m` . . . 13
4.7.3 `models/scheil_model.m` . . . 13
4.7.4 `models/limited_liquid_model.m` . . . 14

4.7.5 `models/clyne_kurz_model.m` . . . 14
4.7.6 `models/won_thomas_model.m` . . . 14
4.8 Template versus authored sections . . . 14
4.9 Differences from the browser version . . . 14

5 Verification **15**
5.1 What was checked . . . 15
5.2 Reference values . . . 15
5.3 How to reproduce the checks . . . 15

6 Assumptions and limitations **16**

7 Quick start **17**

8 Roadmap **17**

A Function reference **18**

B Source of the standalone model functions **18**

C Glossary **26**

D References and source material **26**

1 Introduction and scope

1.1 Purpose

The project is a teaching tool for the solute-redistribution problem in solidification. It answers a single question visually:

> *Given an Al–4.8 wt.% Cu alloy freezing from the liquid, how do the solid and liquid compositions evolve as the solid fraction increases, and how does that answer change when the assumptions about diffusion and mixing change?*

Four classical limits are implemented side by side because each one removes a different assumption:

1. **Equilibrium (lever rule)** — complete diffusion in both phases.
2. **Scheil–Gulliver** — no solid diffusion, perfectly mixed liquid.
3. **No solid diffusion with limited liquid diffusion** — a diffusion boundary layer builds ahead of a planar interface.
4. **Clyne–Kurz** — finite back diffusion in the solid with a mixed liquid, optionally with the Won–Thomas coarsening correction.

The temperature slider adds a second axis of exploration. It moves the equilibrium tie-line and, while coupling is enabled, rescales the partition coefficient and the two diffusivities so that the concentration curves respond to temperature.

1.2 Deliverables

File	Role
`index.html`	Self-contained browser app: layout, styling, four models, SVG charts, animations, temperature coupling. No external dependencies.
`fs_slider.m`	MATLAB app (`uifigure`) reproducing the models, controls, charts and animations.
`models/equilibrium_model.m`	Lever-rule composition, scalar or vector.
`models/scheil_model.m`	Scheil–Gulliver composition and average solid.
`models/limited_liquid_model.m`	Mass-conserving solid profile plus liquid boundary layer.
`models/clyne_kurz_model.m`	Back-diffusion model with optional direct β override.
`models/won_thomas_model.m`	Coarsening-corrected Clyne–Kurz.
`README.md`	Full model documentation and control reference.
`SITE\_OVERVIEW.md`	Plain-language guide to the deployed site.
`report/TECHNICAL\_REPORT.tex`	This report (source).
`preview.png`	Screenshot used in the README and this report.

1.3 What is template and what is authored

Because the graphical scaffolding and the physics are easy to confuse in a single-file app, the project separates them explicitly. The same separation is repeated as banner comments inside `fs_slider.m` and is summarised below.

Provenance of the code

- **UI template (adapted):** the `uifigure`/`uigridlayout` arrangement, the widget styling, the axes layout and the general slider-app skeleton follow the course's MATLAB slider template. They were restyled and extended with extra panels, but they contain no solidification physics.
- **Authored for this project:** all solute-redistribution equations, the four model implementations, the temperature-coupling fits and Arrhenius scalings, the mass-conserving finite-difference integration of the limited-liquid model, the chart data preparation, the animation logic, the browser implementation and all documentation.
- The browser app has no template: its HTML, CSS, JavaScript and SVG rendering were written for this project.

1.4 How to read this report

Section 2 derives every equation. Section 3 describes the website; Section 4 describes the MATLAB app and the five standalone functions in plain language. Section 5 lists the checks that were run. Section 6 collects the assumptions and limits in one place, and Section 7 is a quick-start guide. The appendices contain the full source of all five model functions, a function reference and a glossary.

A four-page condensed version for submission (assumptions, material data and sources, equations, results, limitations) is in `report/TECHNICAL\_SUMMARY.pdf`.

2 Physical model and notation

2.1 The one-dimensional solidification problem

A bar of nominal composition C_0 (wt.% Cu in Al) is frozen from one end. At any instant a fraction f_s is solid and $1 - f_s$ is liquid. The interface is assumed planar and to advance normal to itself. Two relations are used everywhere:

$$\text{local equilibrium at the interface:} \qquad C_S^* = k\, C_L^*, \tag{1}$$

$$\text{solute conservation over the whole bar:} \qquad f_s\, \overline{C_S} + (1 - f_s)\, \overline{C_L} = C_0. \tag{2}$$

Here k is the partition coefficient, C_S^* and C_L^* are the interface compositions, and bars denote averages over each phase. The models differ only in how they close the relation between the interface value and the average — that is, in what they assume about diffusion.

2.2 Equilibrium: the lever rule

Complete diffusion in both phases keeps each phase uniform, so the interface values equal the averages. Substituting $C_S^* = kC_L^*$ into Eq. (2) with $C_S = C_S^*$ and $C_L = C_L^*$ gives

$$f_s\, kC_L + (1 - f_s)\, C_L = C_0 \quad \Longrightarrow \quad C_L = \frac{C_0}{1 - (1 - k) f_s}, \tag{3}$$

and therefore

$$\boxed{C_S = \frac{kC_0}{1 - (1 - k) f_s}}, \qquad \boxed{C_L = \frac{C_0}{1 - (1 - k) f_s}}. \tag{4}$$

Symbol	Meaning	Default (Al–4.8 wt.% Cu)
C_0	nominal composition	4.8 wt.%
k	partition coefficient C_S^*/C_L^*	0.173 at 648 °C
f_s	solid fraction	slider, 0–1
T	interface temperature	648 °C
T_m	melting point of pure Al	660.3 °C
T_{eut}	eutectic / solidus temperature	548 °C
D_S	solid diffusivity	$4.8 \times 10^{-15}\,\mathrm{m^2/s}$
D_L	liquid diffusivity	$2.4 \times 10^{-9}\,\mathrm{m^2/s}$
v	growth velocity	$20\,\mu\mathrm{m/s}$
t_f	local solidification time	100 s
λ_2	secondary dendrite-arm spacing	$20\,\mu\mathrm{m}$
α_C	Won–Thomas coarsening constant	0.10
L	characteristic length for δ/L	1 mm

Table 1: Notation and the reference parameter set.

At $f_s = 0$ this returns $C_S = kC_0$ and $C_L = C_0$; at $f_s = 1$ both phases reach C_0. This is the equilibrium reference limit. For $k < 1$ the denominator vanishes beyond $f_s = 1$, so the model is only evaluated on $[0, 1]$.

2.3 Scheil–Gulliver: no solid diffusion, mixed liquid

With $D_S = 0$ the solid retains the composition it had when it formed, while the liquid stays uniform. Considering an increment of solid df_s:

$$(C_L - C_S)\,df_s = (1 - f_s)\,dC_L \quad \Longrightarrow \quad \frac{dC_L}{C_L} = (k-1)\frac{df_s}{1 - f_s}. \tag{5}$$

Integrating from $(f_s, C_L) = (0, C_0)$:

$$\boxed{C_L = C_0(1 - f_s)^{k-1}}, \qquad \boxed{C_S^* = kC_0(1 - f_s)^{k-1}}. \tag{6}$$

The average solid composition follows from Eq. (2):

$$\overline{C_S} = \frac{C_0\left[1 - (1 - f_s)^k\right]}{f_s}, \qquad \overline{C_S} \to kC_0 \text{ as } f_s \to 0. \tag{7}$$

For $k < 1$ both interface compositions diverge as $f_s \to 1$; the app caps the plotted range and flags the singularity rather than drawing a finite artefact.

2.4 No solid diffusion with limited liquid diffusion

This is the Tiller–Jackson–Rutter–Chalmers picture. There is no solid diffusion and no convective mixing, so rejected solute can only spread by diffusion in the liquid. Ahead of a planar interface moving at v, the steady boundary layer is

$$C_L(z) = C_0 + (C_L^* - C_0)\,e^{-z/\delta}, \qquad C_L(0) = C_L^* = \frac{C_0}{k}, \qquad \boxed{\frac{\delta}{L} = \frac{D_L}{vL}}, \tag{8}$$

where z is measured ahead of the interface and $L = 1\,\mathrm{mm}$ is the characteristic length. At steady state the solid grows at $C_S = C_0$ and $C_L^* = C_0/k$.

Initial and final transients. A pure exponential is not sufficient over the whole bar: the first solid is enriched relative to kC_0, and near the end of the bar the boundary layer is squeezed against the mould wall, which produces a final spike. The implementation therefore integrates the solid profile directly from local equilibrium plus global solute conservation.

Let $x \in [0, f_s]$ be the solidified coordinate and $S(x) = C_S(x)/C_0$ the normalised solid composition. Define the cumulative solute

$$I(f_s) = \int_0^{f_s} S(x)\, dx. \tag{9}$$

The solute still in the liquid, expressed as an equivalent solid fraction, is $f_s - I(f_s)$. If the liquid boundary layer occupies the remaining length $\ell = 1 - f_s$ and must absorb that excess uniformly, then

$$\frac{C_L^*}{C_0} = 1 + \frac{f_s - I(f_s)}{\delta\,(1 - e^{-\ell/\delta})}. \tag{10}$$

Local equilibrium then gives $S(f_s) = k\, C_L^*/C_0$. The code solves Eqs. (10) and (1) by fixed-point iteration on a uniform grid ($N = 2200$ intervals in the browser, $N = 1400$ in MATLAB) with the trapezoidal rule for I:

$$I_i = I_{i-1} + \tfrac{1}{2}\,(S_{i-1} + S_i)\,\Delta x, \qquad S_i^{(m+1)} = k\left[1 + \frac{x_i - I_i^{(m)}}{\delta\,(1 - e^{-(1-x_i)/\delta})}\right], \tag{11}$$

with eight iterations per grid point and $S_0 = k$. This reproduces the initial transient, the $C_S = C_0$ plateau and the final spike in one mass-conserving sweep. The interface values are obtained by interpolation at the requested f_s; the liquid profile is then Eq. (8) with $C_L^* = C_S^*/k$.

Why the fixed point converges. The right-hand side depends on S only through the running integral, and each grid step changes I by $O(\Delta x)$. The map is therefore a contraction for small Δx, and eight sweeps reduce the residual to round-off for the grid sizes used here. The solver also clamps $1 - x$ to 10^{-9} so the exponential near the wall cannot divide by zero.

2.5 Clyne–Kurz: finite back diffusion

The liquid is fully mixed, but the solid retains a finite amount of back diffusion. The dimensionless Fourier number uses half the arm spacing as the diffusion distance:

$$\alpha = \frac{D_S t_f}{X^2} = \frac{4 D_S t_f}{\lambda_2^2}, \qquad X = \frac{\lambda_2}{2}. \tag{12}$$

Clyne and Kurz replace the Brody–Flemings parameter by

$$\boxed{\beta = 2\alpha\left(1 - e^{-1/\alpha}\right) - e^{-1/(2\alpha)}}, \tag{13}$$

which is bounded between 0 and 1. The liquid composition is the generalised back-diffusion form

$$\boxed{\frac{C_L}{C_0} = [1 - (1 - \beta k) f_s]^{\frac{k-1}{1-\beta k}}}, \tag{14}$$

with $C_S^* = kC_L$ and

$$\overline{C_S} = \frac{C_0 - (1 - f_s) C_L}{f_s}. \tag{15}$$

Two limits connect the model to the others:

- $\beta \to 0$ recovers Scheil–Gulliver, Eq. (6).
- $\beta \to 1$ recovers the equilibrium lever rule, Eq. (4).

The implementation handles $k = 1$ and $1 - \beta k = 0$ as special cases (Eq. (14) becomes $C_L = C_0 e^{(1-k)f_s}$) so that no 0/0 is ever evaluated. A user may also supply β directly as a sensitivity variable; in that mode α only affects the reported diagnostics.

2.6 Won–Thomas coarsening correction

Won and Thomas approximate dendrite coarsening by replacing the Fourier number:

$$\boxed{\alpha^{+} = 2\,(\alpha + \alpha_C)}, \qquad \alpha_C = 0.10, \tag{16}$$

and then evaluating Eq. (13) with α^{+}. The factor of two extends the result to columnar dendrites and α_C accounts for the smaller arm spacing present before coarsening is complete. This is an approximate correction, not a full coarsening model, and the original paper's empirical arm-spacing fits are steel-specific and are deliberately not applied.

2.7 Temperature coupling

The notes assume constant k, D_L and liquidus slope m. The interactive temperature knob is an explicit extension: it couples T to the material parameters through standard Al–Cu phase-diagram data and Arrhenius diffusion, so that moving the slider changes the concentration curves.

Liquidus and solidus fits. Two quadratics pass through the phase-diagram anchors

$$\text{liquidus: } (0, 660.3),\ (4.8, 648),\ (33, 548), \tag{17}$$

$$\text{solidus: } (0, 660.3),\ (0.8304, 648),\ (5.65, 548), \tag{18}$$

with the composition in wt.% and the temperature in °C:

$$T = -0.0298060391\, C_L^2 - 2.41943101\, C_L + 660.3, \tag{19}$$

$$T = -1.05070285\, C_S^2 - 13.93963508\, C_S + 660.3. \tag{20}$$

For a given T the physical root is

$$C(T) = \frac{-b - \sqrt{b^2 - 4a(c - T)}}{2a}, \qquad a < 0,\ b < 0,\ c = 660.3, \tag{21}$$

and $C = 0$ when $T \geq c$ or the discriminant is negative. The partition coefficient then follows from the phase-diagram ratio:

$$\boxed{k(T) = \frac{C_S(T)}{C_L(T)}}, \qquad k(T) = k_{\text{ref}} \text{ if } C_L < 10^{-8}. \tag{22}$$

Arrhenius diffusivities. Both diffusivities are anchored at $T_{\text{ref}} = 648\,°\text{C}$:

$$D_S(T) = D_{S,\text{ref}} \exp\left[-\frac{Q_S}{R_g}\left(\frac{1}{T + 273.15} - \frac{1}{T_{\text{ref}} + 273.15}\right)\right], \tag{23}$$

$$D_L(T) = D_{L,\text{ref}} \exp\left[-\frac{Q_L}{R_g}\left(\frac{1}{T + 273.15} - \frac{1}{T_{\text{ref}} + 273.15}\right)\right], \tag{24}$$

with $D_{S,\text{ref}} = 4.8 \times 10^{-15}\,\text{m}^2/\text{s}$, $D_{L,\text{ref}} = 2.4 \times 10^{-9}\,\text{m}^2/\text{s}$, $Q_S = 130\,\text{kJ/mol}$, $Q_L = 30\,\text{kJ/mol}$ and $R_g = 8.314462618\,\text{J mol}^{-1}\text{K}^{-1}$.

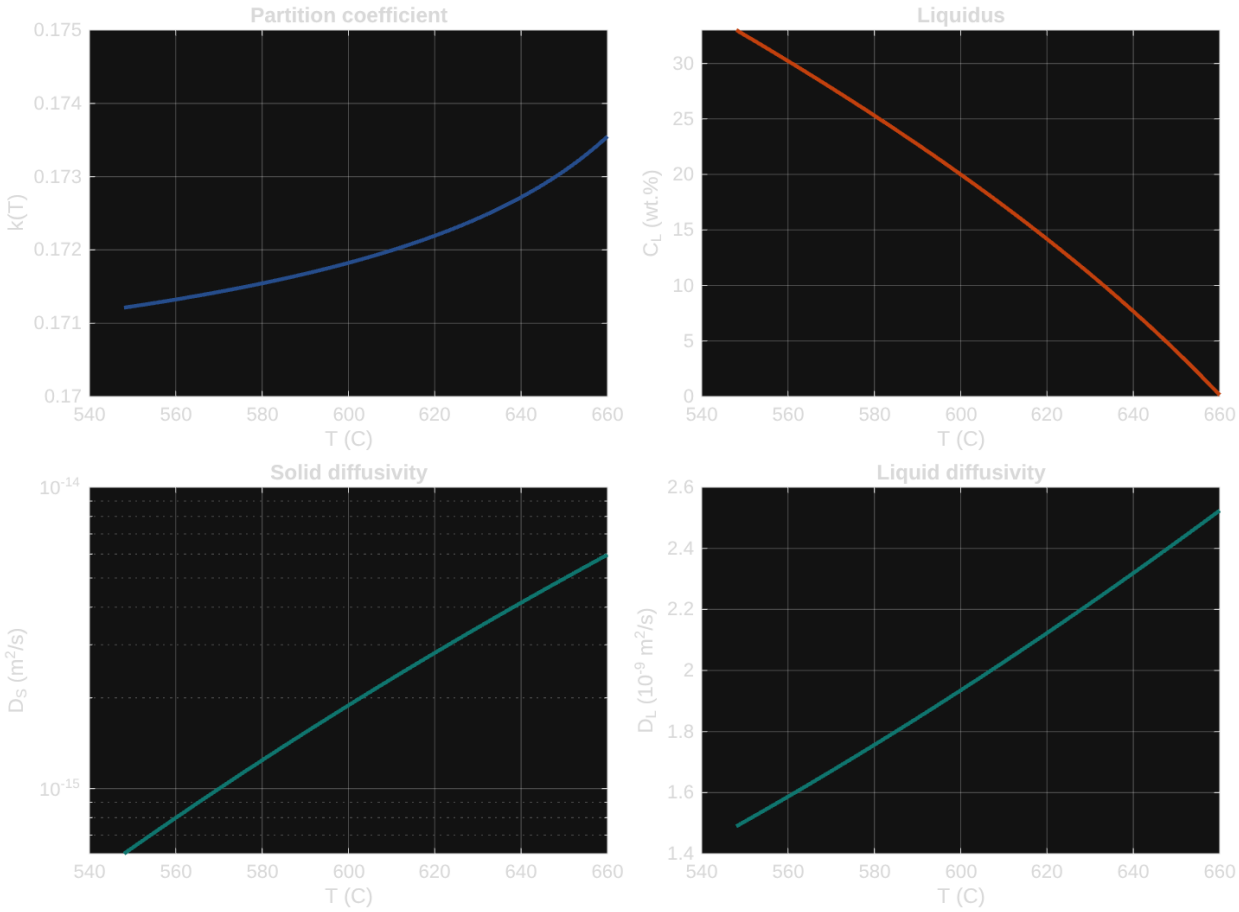


Figure 1: Temperature coupling over the slider range. Top left: $k(T)$ is almost flat for Al–Cu (0.1712 to 0.1735). Top right: liquidus composition from Eq. (19). Bottom row: Arrhenius $D_S(T)$ and $D_L(T)$ from Eqs. (23)–(24).

Consequences for the demo. Figure 1 makes an important point explicit: $k(T)$ moves only by about 1.3% across 548–660 °C. Because the normalised equilibrium and Scheil curves depend only on k, they barely change with temperature. The visible temperature effect is dominated by $D_S(T)$, which is about 7.9 times smaller at 548 °C than at 648 °C, and acts through α in Clyne–Kurz and through δ/L in the limited-diffusion model.

The notes' spatial model. Pages 7–9 of the handwritten notes go further and combine the steady profile with a thermal gradient:

$$C_L(z) = C_0\left[1 + \frac{1-k}{k} e^{-Rz/D_L}\right], \tag{25}$$

$$T_i = T_m - \frac{mC_0}{k}, \tag{26}$$

$$T_L(z) = T_i + mC_0\frac{1-k}{k}\left[1 - e^{-Rz/D_L}\right], \tag{27}$$

$$T(z) = T_i + G_L z, \tag{28}$$

$$\frac{G_L}{R} \geq \frac{mC_0(1-k)}{kD_L}. \tag{29}$$

This spatial model is *not* what the temperature slider evaluates: it requires a thermal gradient G_L and a distance axis, neither of which is an input to the current app. It is documented here because it is the notes' temperature model and because Eq. (29) is the natural next extension (Section 8).

The interactive slider instead uses the tie-line/fit route above, which is well posed for the available controls.

2.8 Numerical edge cases

Case	Implementation
$k = 1$	Clyne–Kurz returns $C_L = C_0$; Scheil and lever rule stay finite.
$1 - \beta k = 0$	Eq. (14) evaluated as $C_L = C_0 e^{(1-k)f_s}$.
$\alpha \to 0$	$\beta \to 0$ (Scheil limit); $\alpha \to \infty$ gives $\beta \to 1$.
$f_s \to 0$	Averages evaluated analytically ($\overline{C_S} = kC_0$), not by 0/0.
$f_s \to 1$	Singularities flagged; limited-liquid returns `Inf`, not a fake finite value.
$T \geq T_m$	Fully liquid: tie-line shown as dashes/blank, k held at k_{ref} for the limiting curves.
$C_L < 10^{-8}$	$k(T)$ falls back to k_{ref} instead of dividing by zero.
discriminant < 0	Fit root returns 0 instead of a complex number.
$1 - x \leq 0$	Clamped to 10^{-9} inside the boundary-layer solver.

Table 2: Guarded cases in the JavaScript and MATLAB implementations.

3 The browser application

3.1 Architecture

`index.html` is a single file with inline CSS and one JavaScript IIFE. It has no frameworks, no build step, no CDN, no web fonts and no network requests. It works from the filesystem (`file://`) and from GitHub Pages.

The script is organised in five layers:

1. **Data and state.** An `ALLOY` constant holds the reference material; a single mutable `state` object holds every input (f_s, T, k, C_0, model, view, v, D_L, D_S, t_f, λ_2, α_C, the two toggles and the direct β).
2. **Model functions.** Pure functions of the state: `cs()`, `cl()`, `csAvg()`, `clAvg()`, `ckCLNorm()`, `ckAvgNorm()`, `limitedSolidProfileNorm()`, `limitedLiquidProfileNorm()`, plus the temperature helpers `liquidusC()`, `solidusC()`, `kAtT()`, `dsAtT()`, `dlAtT()`.
3. **Readout sync.** `syncTemperature()`, `syncParameterSliders()`, `syncModelUI()`, `updateParamHints()`.
4. **Rendering.** `buildChart()` and `buildSpatialChart()` rebuild two SVG groups from scratch on every update using a small `svgEl()` helper.
5. **Events.** Sliders, number fields, model/view buttons and play buttons call `setState(patch)`, which merges the patch and runs `update()`.

The single entry point is `update()`: it syncs the derived properties and labels, recomputes the compositions, redraws both charts and refreshes the mass-balance check. Every interaction goes through it, so the readout and the charts can never disagree.

3.2 State and update flow

user input → `setState(patch)` → `Object.assign(state, patch)` → `update()` → `syncTemperature()` → `syncParameterSliders()` → model evaluation → `buildChart()` + `buildSpatialChart()`

`syncTemperature()` is the only place that writes derived material properties back into `state`. When `state.tempLinked` is true it sets `state.k = kAtT(T)`, `state.ds = dsAtT(T)` and `state.dl = dlAtT(T)`, and mirrors them into the number fields. When it is false it only refreshes the readouts, so a manual k, D_S or D_L survives a temperature change. The two D fields are disabled while coupling is on, because a value typed there would be overwritten on the next frame.

3.3 Controls and override semantics

Control	Behaviour
f_s slider	0–1 in steps of 0.001; the badge always shows three decimals.
T slider	548–700 °C in steps of 1 °C; the badge is a fixed-width element (84 px) showing an integer, so the layout cannot shift as the value changes.
k slider	logarithmic 0.01–100 over 1000 internal steps; the number field is always editable.
C_0 slider	the same logarithmic mapping, default 4.8 wt.%.
Model buttons	four-way switch; each model has its own blurb and equation card.
View switch	*Auto zoom* rescales the along-bar chart with f_s; *Fixed scale* freezes axes and grid.
Play buttons	one per parameter; see Section 3.5.
Coupling box	*Derive k, D_s and D_L from T*, checked by default.

Table 3: Browser controls.

Three interactions count as a manual override of the temperature coupling: dragging the k slider, typing in the k number field, and starting the k animation. Each one clears the coupling flag and unchecks the box. Re-checking the box re-anchors all three derived properties at the current temperature. The k label and the tie-line label switch between `k(T)` and `k` so the readout always names the value the curves are actually using.

3.4 Charts

Both charts are hand-built SVG. The main chart uses a 860×500 viewBox with margins $\{24, 30, 66, 82\}$ (top, right, bottom, left). It plots normalised composition C/C_0 against f_s:

- solid curve: $C_S(f_s)/C_0$;
- liquid curve: $C_L(f_s)/C_0$ (dashed except for the limited model, where the boundary-layer profile is drawn);
- for Clyne–Kurz, an additional average-solid curve;
- horizontal dashed tie-line references at $\pm 4\%$ of the selected solute fraction, labelled C_L^* and C_S^*;
- a vertical interface marker with a solid–liquid connector;
- a shaded band for the current solid fraction.

The along-bar chart uses a 860×320 viewBox with a 30 px bar above the plot area. Its x axis is position $x/L \in [0, 1]$, tinted teal behind the solid and amber ahead of the interface. The solid profile is model-dependent: Scheil shows $k(1-x)^{k-1}$, the limited model shows the integrated profile, and Clyne–Kurz shows the back-diffused average as a homogenised solid. The liquid side shows either a flat composition or the decaying boundary layer.

Axes are recomputed on every draw. The normalised y limit is the maximum of the current curve values times a 6–12 % headroom, capped at 25 so that the Scheil singularity cannot make the plot unreadable. In fixed-scale mode the limit is the model-wide value, which keeps the grid identical while f_s moves. Ticks come from a `ticksFor()` helper that chooses a round interval for roughly six ticks.

3.5 Animation

Each play button sweeps its parameter from one end of its range to the other and back. The animation is driven by `requestAnimationFrame` at a rate of 0.18 of the full range per second, with the frame step capped at 50 ms so a background tab cannot jump. Only one animation runs at a time; starting another stops the previous one. Any manual slider or number-field edit stops the animation. The temperature animation snaps to whole degrees and the k/C_0 animations use the same logarithmic mapping as their sliders, so the animation path matches the slider path exactly. `prefers-reduced-motion` disables the CSS transitions used for the badge and panel fades.

3.6 Responsive layout and accessibility

The page is a two-column layout (controls + readout on the left, charts on the right) above 900 px. Below 900 px it becomes a single column with the chart first; below 560 px the stat boxes stack. Every input has an `aria-label`, each chart has a `<title>` and `<desc>`, model and view buttons are `role="group"` with `aria-pressed`, and the play buttons announce *Animate*/*Pause*. Colours were checked for contrast on the light background. No information is carried by colour alone: solid and liquid are distinguished by line style and labels as well.

3.7 Performance

The limited-liquid model is the only expensive one. Its solid-profile sweep is cached on the key `k|delta`, so dragging f_s or T without changing either parameter reuses the grid. The chart is rebuilt as a small number of SVG nodes on each update, which is cheap at this size. All model functions are pure and allocation-light; there is no global DOM query inside the render loop (the elements are cached once in an `els` map).

4 The MATLAB application

4.1 Requirements and how to run

```
% Put the project folder on the path, then:
addpath('models');
fs_slider
```

The app requires MATLAB R2020a or newer (`uifigure`, `uislider`, `uidropdown`, `uigridlayout`). It uses base MATLAB only: no toolboxes, no external files, no network. It is self-contained in

`fs\_slider.m`; the `models/` functions are optional stand-alone versions of the same equations and are linked from the documentation.

4.2 Architecture: one function, nested callbacks

`fs\_slider.m` is a single top-level function with no return value. All widgets, state variables and helper functions live in its workspace; the callbacks are *nested functions*, so they read and write those variables directly. This is the simplest possible MATLAB app architecture and is the main reason the file reads linearly from top to bottom:

1. **Parameters** (lines 46–72): material constants and defaults.
2. **Part I — UI template** (from `\%\% UI`): figure, grid layouts, widgets, styling, axes and plot objects. This is the adapted course template.
3. **Part II — authored logic**: callbacks, animation, temperature coupling, model equations, plotting and the update cycle.

There is one shared clock for animation (`animTimer`, a MATLAB `timer` at 20 fps) and one entry point for refreshes: `updateDisplay()`. Every callback ends by calling it, so the MATLAB app has the same single-source-of-truth property as the browser app.

4.3 The UI template part

The figure is a 1180×1000 `uifigure` with a three-row `uigridlayout`: header, controls/charts, and a status strip. The control column is a 15-row $\times$ 3-column grid that stores one logical control per row, so the vertical order of the interface is visible at a glance in the `RowHeight` vector. Rows are shown or hidden by setting their height to zero in `updatePanelLayout()`; this is how the model-specific parameter panels change without rebuilding the figure.

Row	Control
1	model dropdown
2	temperature slider (T)
3–4	tie-line label
5	*Derive k, D_s and D_L from T* checkbox
6	solid-fraction slider (f_s)
7–8	k slider, number field and play button
9–10	C_0 slider, number field and play button
11	model-specific panel (limited liquid or Clyne–Kurz)
12–15	status strip and footer labels

Table 4: Control-grid row map in `fs\_slider.m`.

The two axes are created once and reused; `updateDisplay()` clears only their children. The main axes shows composition versus f_s; the second axes shows the along-bar profile and is re-titled with the current solid fraction. Line objects for the curves, the tie-lines and the interface marker are created once and their `XData`/`YData` are updated, which is the recommended MATLAB pattern and avoids flicker.

4.4 Callbacks and the update cycle

widget callback → `stopAnim()` → (optional override / rounding) → `updateDisplay()` → `syncTemperature()` → model evaluation → axes + readout

`updateDisplay(fsRequested)` is the workhorse. It refreshes the derived properties, evaluates the selected model, computes the averages and the mass balance, builds the two plots and writes the readout text. `syncTemperature()` writes derived k, D_S and D_L into the numeric fields while coupling is on and disables the two diffusivity fields; it leaves k editable because typing there is the intended override gesture.

4.5 Animation

The four triangular buttons toggle a timer-driven sweep. `toggleAnim()` records the target and the current normalised position, sets the direction, and starts `animTimer` if it is not already running. `animStep()` advances the position by 0.05×0.18 per tick, reflects it at the ends, and calls the same setters a user would. Starting a k animation clears the coupling checkbox, matching the browser behaviour. `stopAnim()` deletes nothing: it stops the timer and clears the target, so the timer object is reused. The figure's `CloseRequestFcn` stops and deletes the timer before deleting the figure, which prevents orphaned timers from keeping MATLAB busy.

4.6 Plotting and panels

The MATLAB plots are deliberately simpler than the browser SVG but follow the same data pipeline: normalised composition, model-specific solid and liquid curves, two dashed tie-line references, an interface marker and a shaded solid-fraction band. The along-bar axes draws the solid profile on $x/L \in [0, 1]$ and the liquid boundary layer ahead of the interface. The view dropdown switches between auto and fixed limits exactly as in the browser.

Parameter panels are toggled by row height, not by visibility alone, because a zero-height row collapses cleanly in `uigridlayout`. The limited-liquid panel shows v, D_L and the derived δ/L; the Clyne–Kurz panel shows D_s, t_f, λ_2, α_C, the direct-β override and the Won–Thomas checkbox. `updatePanelLayout()` is called on model changes and from the update cycle, so the panel can never be out of step with the model.

4.7 File-by-file guide

4.7.1 `fs\_slider.m` — the app

The header block states the author, the four model equations, the temperature coupling, how to run, the MATLAB version requirement and the default material. The file then defines parameters, builds the UI, and implements callbacks, animation, coupling, models and plotting. Section banners mark the boundary between the template and the authored logic. It has no dependencies beyond base MATLAB and is the only file needed to run the app.

4.7.2 `models/equilibrium\_model.m`

Signature `[Cs, Cl, CsAvg, ClAvg] = equilibrium_model(k, c0, fs)`. Evaluates Eq. (4) vectorised over `fs`. `CsAvg` and `ClAvg` are aliases of `Cs` and `Cl` because both phases are uniform. Example: `equilibrium_model(0.173, 4.8, 0.5)` returns $C_S = 1.416$, $C_L = 8.184$ wt.%.

4.7.3 `models/scheil\_model.m`

Signature `[Cs, Cl, CsAvg, ClAvg] = scheil_model(k, c0, fs)`. Evaluates Eqs. (6)–(7). The average is computed analytically and the $f_s \to 0$ limit is handled separately rather than by dividing

by zero. `ClAvg` equals `Cl` because the liquid is mixed.

4.7.4 `models/limited\_liquid\_model.m`

Signature `out = limited_liquid_model(k, c0, v, DL, fs, N, L)`. Returns a struct with the solid grid, solid profile, δ, interface values, averages, the liquid grid and the boundary-layer profile. It performs the fixed-point sweep described in Section 2.4 (default $N = 1400$, $L = 1\,\text{mm}$) and returns `Inf` at the singular end point instead of a finite artefact. Example: `out = limited_liquid_model(0.173,4.8,20e-6,2.4e-9,0.5)` gives $\delta/L = 0.12$.

4.7.5 `models/clyne\_kurz\_model.m`

Signature `out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs, alphaC, useWT, betaOverride)`. Returns α, α_{eff}, β, C_L, C_S^*, $\overline{C_S}$ and $\overline{C_L}$. It implements Eqs. (13)–(14) with the special cases $k = 1$ and $1 - \beta k = 0$. When `betaOverride` is supplied, β is clamped to $[0, 1]$ and used directly; α and the Won–Thomas flag then only affect the diagnostics.

4.7.6 `models/won\_thomas\_model.m`

Signature `out = won_thomas_model(k, c0, Ds, tf, lambda2, fs, alphaC)`. A thin wrapper that calls `clyne\_kurz\_model(..., true)` and exposes `alphaWT`. Default $\alpha_C = 0.1$.

4.8 Template versus authored sections

Part of the file	Origin	Contains physics?
Parameter block	authored	defaults only
`\%\% UI` to end of widget creation	course template, adapted/restyled	no
`updatePanelLayout`, `syncParamSliders`	template pattern, extended	no
Callbacks and animation	authored	no (glue)
Temperature coupling	authored	yes
Model functions (`isScheil`, `ckCLNorm`, `limitedProfile`, ...)	authored	yes
`updateDisplay` plotting data	authored	yes
Readout text	authored	descriptive

Table 5: Provenance map for `fs\_slider.m`. The file repeats this split as banner comments so a reader can see it without this report.

4.9 Differences from the browser version

The physics is identical; only the plumbing differs. The browser uses $N = 2200$ grid intervals in the limited model and MATLAB uses $N = 1400$; both are converged to plotting accuracy. The browser draws SVG by hand, MATLAB uses `plot`/`fill`. The browser animates with `requestAnimationFrame`, MATLAB with a 20 fps `timer`. The browser uses logarithmic sliders with 1000 internal steps, MATLAB uses its native `uislider` with a position→value mapping. The MATLAB k and C_0 number fields use `ValueDisplayFormat` and the same override rules as the website.

5 Verification

5.1 What was checked

Check	Method	Result
JavaScript syntax	`node -check` on the extracted script	pass
Browser UI and coupling	headless Chrome (CDP), assertions on DOM values	pass
Browser models/views/animation	all four models, both views, all play buttons	pass
Temperature badge	integer text and constant 84 px width at 548, 549, 640, 648, 700	pass
Override semantics	drag/type/animate k clears coupling; re-check re-anchors	pass
MATLAB static analysis	`checkcode('fs\_slider.m','-string')`	no messages
MATLAB app regression	scripted `uifigure` interaction	pass
Browser/MATLAB parity	$k(640)$, $D_S(640)$, $D_L(640)$, tie-line values	agree
Deployed page	fetch of the GitHub Pages URL	updated build served

Table 6: Verification performed before each release.

5.2 Reference values

At the default state ($f_s = 0.5$, $k = 0.173$, $C_0 = 4.8$ wt.%) the equilibrium model returns $C_S = 1.416$ and $C_L = 8.184$ wt.%. The temperature coupling returns the values in Table 7.

T (°C)	C_L (wt.%)	C_S (wt.%)	$k(T)$	D_S (m^2/s)
548	33.00	5.650	0.1712	6.07×10^{-16}
600	20.00	3.436	0.1718	1.89×10^{-15}
640	7.67	1.324	0.1727	4.14×10^{-15}
648	4.80	0.830	0.1730	4.80×10^{-15}
660	0.12	0.021	0.1735	5.97×10^{-15}

Table 7: Values reproduced by both front ends. D_L moves from 1.49×10^{-9} at 548 °C to 2.96×10^{-9} at 700 °C.

5.3 How to reproduce the checks

The test scripts are deliberately kept outside the repository so that the deliverable stays small. They use only Node's built-in `assert` and the Chrome DevTools Protocol, plus MATLAB's own `assert`:

```
# syntax
python3 -c "import re;print(re.findall(r'<script>(.*?)</script>',
  open('index.html').read(), re.S)[-1])" > /tmp/app.js
node --check /tmp/app.js

# MATLAB static analysis
matlab -batch "checkcode('fs_slider.m','-string')"
```

The headless tests start Chrome with `-headless=new -remote-debugging-port=9222`, open `index.html`, dispatch real `input` and `change` events, and assert on the DOM. The MATLAB test builds the app, locates the widgets by type and limits, drives the callbacks, and asserts on field values and labels.

6 Assumptions and limitations

Physical assumptions, common to all models

1. One-dimensional, planar interface advancing at constant velocity.
2. Local equilibrium at the interface, $C_S^* = kC_L^*$. The models differ in the diffusion assumptions, not in the interface condition.
3. Constant partition coefficient within a single model evaluation. The temperature coupling deliberately relaxes this as a sensitivity study.
4. Constant densities and no convection, except where a model assumes the liquid is fully mixed.
5. The terminal eutectic is not modelled. The eutectic composition (≈ 33 wt.%) is used only as a phase-diagram anchor.
6. Solute transport is described by Fick's law with a single diffusivity per phase.

Model-specific assumptions

- **Equilibrium:** infinite diffusion in both phases; the result is an ideal reference limit, not a casting prediction.
- **Scheil:** zero solid diffusion and perfect liquid mixing. The interface composition diverges as $f_s \to 1$ for $k < 1$; the app caps the view and flags the singularity.
- **Limited liquid:** no solid diffusion, no liquid mixing. The boundary-layer form is the steady solution; the transient is captured by the mass-conserving sweep. Requires $\delta/L < 1$ for a thin layer; the interface is assumed planar although it may be constitutionally supercooled.
- **Clyne–Kurz:** finite back diffusion, mixed liquid. It is an analytical approximation and does not resolve a spatial solid profile. The Won–Thomas correction is approximate and its empirical arm-spacing fits are steel-specific, so they are not used.

Temperature coupling

- The quadratic liquidus/solidus fits and the activation energies are standard Al–Cu approximations. They are an *extension* beyond the notes, which assume constant k, D_L and m.
- The fits are extrapolated across 548–700 °C. They should be used near the alloy composition, not as a full phase-diagram prediction.
- $k(T)$ is nearly flat for Al–Cu, so the equilibrium and Scheil *normalized* curves barely respond to temperature; most of the visible effect is through $D_S(T)$ and $D_L(T)$.
- The notes' spatial model ($T_L(z)$, G_L, constitutional supercooling) is documented but not driven by the slider, because the app has no thermal-gradient or distance input.

Numerical assumptions

The limited-liquid sweep uses a uniform grid and eight fixed-point iterations per node; both are converged for plotting. The Scheil and Clyne–Kurz singularities are capped or flagged. The browser caps the y range at 25 so a divergence cannot destroy the scale. All displayed values are rounded for legibility; the underlying computation is double precision.

7 Quick start

Browser

```
cd fs_slider_project
python3 -m http.server 8000
# open http://localhost:8000
```

or simply open `index.html` directly. The live copy is at https://1clan.github.io/mse670-solute-redistribution/.

MATLAB app

```
cd fs_slider_project
fs_slider
```

Stand-alone model functions

```
addpath('models');
[Cs, Cl] = equilibrium_model(0.173, 4.8, 0.5);
out = scheil_model(0.173, 4.8, linspace(0,1,50));
out = limited_liquid_model(0.173, 4.8, 20e-6, 2.4e-9, 0.5);
out = clyne_kurz_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.5);
out = won_thomas_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.5);
```

This report

```
cd report
tectonic TECHNICAL_REPORT.tex
# produces TECHNICAL_REPORT.pdf
```

8 Roadmap

The most natural extension is the notes' spatial temperature model. It needs only two additions: a thermal-gradient input G_L and a temperature axis. With $T_i = T_m - mC_0/k$ and the profile of Eq. (29), the app could plot $T_L(z)$ and $T(z) = T_i + G_L z$, shade the region where $T < T_L$, and show the constitutional-supercooling criterion $G_L/R \geq mC_0(1-k)/(kD_L)$ as a pass/fail badge. That would make temperature reshape the *profile*, not just the tie-line, while keeping the notes' constant-property assumptions inside the spatial model.

Other extensions, in rough order of value: a CALPHAD-backed $k(T)$ for multi-component alloys; convection or a stirring parameter in the limited-diffusion model; a time-dependent growth velocity; and export of the current curves as CSV for comparison with experimental data.

A Function reference

Function	Signature	Returns
`equilibrium\_model`	`(k, c0, fs)`	`[Cs, Cl, CsAvg, ClAvg]`
`scheil\_model`	`(k, c0, fs)`	`[Cs, Cl, CsAvg, ClAvg]`
`limited\_liquid\_model`	`(k, c0, v, DL, fs, N, L)`	struct with profiles, δ, interface and average values
`clyne\_kurz\_model`	`(k, c0, Ds, tf, lambda2, fs, alphaC, useWT, betaOverride)`	struct with α, β, compositions
`won\_thomas\_model`	`(k, c0, Ds, tf, lambda2, fs, alphaC)`	struct with `alphaWT` and the Clyne–Kurz fields

All five functions accept vector `fs` where it is meaningful, validate their scalar inputs, and are independent of the GUI. They are the reference implementation used to cross-check the browser.

B Source of the standalone model functions

The five files below are the reference implementation of the equations. They are independent of the GUI, validate their inputs, and accept vector `fs` where meaningful. Each header states the author, the equations and how to run it.

```
function [Cs, Cl, CsAvg, ClAvg] = equilibrium_model(k, c0, fs)
%EQUILIBRIUM_MODEL  Lever-rule solute redistribution (complete diffusion).
%
%   [Cs, Cl, CsAvg, ClAvg] = equilibrium_model(k, c0, fs)
%
%   Complete diffusion in both solid and liquid keeps both phases uniform.
%   Local equilibrium is k = Cs/Cl, and the lever rule gives
%
%       Cs = k*c0 / (1 - (1-k)*fs)
%       Cl =    c0 / (1 - (1-k)*fs)
%
%   Inputs
%       k   partition coefficient, k > 0 (not restricted to k <= 1)
%       c0  nominal composition
%       fs  solid fraction, 0 <= fs <= 1 (scalar or array)
%
%   Outputs
%       Cs      solid composition
%       Cl      liquid composition
%       CsAvg   average solid composition (= Cs for equilibrium)
%       ClAvg   average liquid composition (= Cl for equilibrium)
%
%   Example (Al-4.8 wt.% Cu)
%       [Cs, Cl] = equilibrium_model(0.173, 4.8, 0.5)   % Cs = 1.416, Cl = 8.184
%
%   HOW TO RUN
%       addpath('models')                % from fs_slider_project
```

```
%       [Cs, Cl] = equilibrium_model(0.173, 4.8, 0.5);
%       fs = linspace(0,1,200);
%       [Cs, Cl] = equilibrium_model(0.173, 4.8, fs);   % vectorised
%
%   AUTHOR
%       Prathamesh Deepak Ladhe, MSE 670 -- Solidification
%
%   SEE ALSO
%       scheil_model, clyne_kurz_model, limited_liquid_model.
%       Derivation: report/TECHNICAL_REPORT.pdf, Section 2.2.

den = 1 - (1 - k).*fs;
Cs = (k .* c0) ./ den;
Cl = c0 ./ den;
CsAvg = Cs;
ClAvg = Cl;
end
```

Listing 1: equilibrium_model.m

```
function [Cs, Cl, CsAvg, ClAvg] = scheil_model(k, c0, fs)
%SCHEIL_MODEL  Scheil-Gulliver solute redistribution.
%
%   [Cs, Cl, CsAvg, ClAvg] = scheil_model(k, c0, fs)
%
%   No solid diffusion (D_S = 0) and a perfectly mixed liquid (D_L -> inf),
%   with local equilibrium k = Cs/Cl at the interface:
%
%       Cs = k*c0*(1 - fs)^(k-1)
%       Cl =    c0*(1 - fs)^(k-1)
%
%   The solid is cored. Its average composition follows from the solute
%   balance and is
%
%       CsAvg = c0*[1 - (1 - fs)^k] / fs        (CsAvg -> k*c0 as fs -> 0)
%
%   Inputs
%       k   partition coefficient, k > 0 (not restricted to k <= 1)
%       c0  nominal composition
%       fs  solid fraction, 0 <= fs <= 1 (scalar or array)
%
%   Outputs
%       Cs      interface solid composition
%       Cl      liquid composition
%       CsAvg   average solid composition
%       ClAvg   average liquid composition (= Cl; the liquid is uniform)
%
%   For k < 1 the liquid and interface solid compositions diverge as fs -> 1.
%
%   Example (Al-4.8 wt.% Cu)
%       [Cs, Cl, CsAvg] = scheil_model(0.173, 4.8, 0.5)
%
%   HOW TO RUN
%       addpath('models')                % from fs_slider_project
%       [Cs, Cl] = scheil_model(0.173, 4.8, 0.5);
%       fs = linspace(0,0.99,200);
%       [Cs, Cl, CsAvg] = scheil_model(0.173, 4.8, fs);
%
%   AUTHOR
%       Prathamesh Deepak Ladhe, MSE 670 -- Solidification
%
%   SEE ALSO
%       equilibrium_model, clyne_kurz_model, limited_liquid_model.
%       Derivation: report/TECHNICAL_REPORT.pdf, Section 2.3.
```

```

Cl = c0 .* (1 - fs).^(k - 1);
Cs = k .* Cl;
ClAvg = Cl;

CsAvg = zeros(size(fs));
small = fs < 1e-9;
if isscalar(fs)
    if small
        CsAvg = k*c0;
    else
        CsAvg = c0*(1 - (1 - fs)^k)/fs;
    end
else
    CsAvg(small) = k*c0;
    idx = ~small;
    CsAvg(idx) = c0*(1 - (1 - fs(idx)).^k)./fs(idx);
end
end
```

Listing 2: scheil_model.m

```
function out = limited_liquid_model(k, c0, v, DL, fs, N, L)
%LIMITED_LIQUID_MODEL  No solid diffusion, limited liquid diffusion.
%
%   out = limited_liquid_model(k, c0, v, DL, fs)
%   out = limited_liquid_model(k, c0, v, DL, fs, N, L)
%
%   No solid diffusion (D_S = 0) and no liquid mixing (D_L finite). Solute
%   rejected by the growing solid can only spread by diffusion in the liquid,
%   so a boundary layer builds up ahead of the interface. With local
%   equilibrium at the interface, Cs* = k*Cl*.
%
%   At steady state the interface liquid composition reaches c0/k, the solid
%   grows at c0, and the liquid profile is
%
%       CL(z) = c0 + (Cl* - c0)*exp(-z/(delta/L)),
%       delta/L = DL/(v*L)
%
%   with boundary-layer thickness delta. The initial transient is governed by
%   (delta/L)/k, and near the end of the bar the boundary layer is squeezed
%   against the mold wall, producing a final-transient spike.
%
%   This implementation integrates the solid profile from the local
%   equilibrium condition and the global solute balance
%
%       integral_0^fs (Cs/c0) dx = fs - (1 - fs)*(Cl,avg/c0 - 1)
%
%   so the initial transient, the Cs = c0 plateau, and the final spike are
%   mass-conserving. The liquid boundary layer is reported separately.
%
%   Inputs
%       k    partition coefficient, k > 0 (not restricted to k <= 1)
%       c0   nominal composition
%       v    interface growth velocity (m/s)
%       DL   liquid diffusivity (m^2/s)
%       fs   solid fraction at which to evaluate the interface (default 0.5)
%       N    number of grid intervals (default 1400)
%       L    characteristic length (m, default 1e-3)
%
%   Outputs (struct)
%       out.x           solid position grid, 0 .. 0.9999
%       out.csProfile   solid composition Cs(x) along the bar
%       out.fs          requested solid fraction
```

```
%       out.delta       boundary-layer thickness, delta/L = DL/(v*L)
%       out.CsStar      interface solid composition at fs
%       out.ClStar      interface liquid composition at fs
%       out.CsAvg       average solid composition at fs
%       out.ClAvg       average liquid composition at fs
%       out.z           distance ahead of the interface, 0 .. (1-fs)
%       out.ClProfile   liquid boundary layer CL(z) at fs
%
%   The interface values are singular in the ideal no-mixing limit as
%   fs -> 1. The function returns Inf there rather than a finite artefact.
%
%   Example (Al-4.8 wt.% Cu at 648 C)
%       out = limited_liquid_model(0.173, 4.8, 20e-6, 2.4e-9, 0.5);
%       % default L = 1 mm; delta/L = 0.12
%
%   HOW TO RUN
%       addpath('models')                % from fs_slider_project
%       out = limited_liquid_model(0.173, 4.8, 20e-6, 2.4e-9, 0.5);
%       plot(out.x, out.csProfile)       % solid profile along the bar
%       plot(out.z, out.ClProfile)       % liquid boundary layer
%
%   AUTHOR
%       Prathamesh Deepak Ladhe, MSE 670 -- Solidification
%
%   SEE ALSO
%       equilibrium_model, scheil_model, clyne_kurz_model.
%       Derivation: report/TECHNICAL_REPORT.pdf, Section 2.4.

if nargin < 5 || isempty(fs)
    fs = 0.5;
end
if nargin < 6 || isempty(N)
    N = 1400;
end
if nargin < 7 || isempty(L)
    L = 1e-3;
end
if v <= 0
    error('limited_liquid_model:v','v must be positive.');
end
if DL <= 0
    error('limited_liquid_model:DL','DL must be positive.');
end
if L <= 0
    error('limited_liquid_model:L','L must be positive.');
end

delta = DL/(v*L);

fs = max(0, min(1, fs));
xmax = 0.9999;
dx = xmax/N;
x = (0:N)'*dx;
S = zeros(N+1,1);
I = zeros(N+1,1);
S(1) = k;

for i = 2:N+1
    xx = x(i);
    l = max(1 - xx, 1e-9);
    sPrev = S(i-1);
    s = sPrev;
    for it = 1:8
```

```
        Ii = I(i-1) + 0.5*(sPrev + s)*dx;
        deficit = xx - Ii;
        cStar = 1 + deficit/(delta*(1 - exp(-1/delta)));
        s = k*cStar;
    end
    S(i) = s;
    I(i) = I(i-1) + 0.5*(sPrev + s)*dx;
end

out.x = x;
out.csProfile = c0*S;
out.fs = fs;
out.delta = delta;

if fs >= 1 - 1e-12
    out.CsStar = Inf;
    out.ClStar = Inf;
    out.CsAvg = c0;
    out.ClAvg = Inf;
    out.z = 0;
    out.ClProfile = Inf;
    return;
end

xx = min(fs, xmax);
csStarN = interp1(x, S, xx);
Ii = interp1(x, I, xx);

out.CsStar = c0*csStarN;
out.ClStar = out.CsStar/k;

if fs < 1e-9
    out.CsAvg = k*c0;
    out.ClAvg = c0;
else
    out.CsAvg = c0*Ii/fs;
    out.ClAvg = c0*(1 - Ii)/(1 - fs);
end

out.z = linspace(0, 1 - fs, 140);
out.ClProfile = c0 + (out.ClStar - c0)*exp(-out.z/delta);
end
```

Listing 3: limited_liquid_model.m

```
function out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs, alphaC, useWT, betaOverride)
%CLYNE_KURZ_MODEL  Back-diffusion microsegregation model of Clyne and Kurz.
%
%   out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs)
%   out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs, alphaC, useWT)
%   out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs, alphaC, useWT, betaOverride)
%
%   The liquid is fully mixed, but the solid retains a finite amount of
%   back diffusion. The dimensionless Fourier number is
%
%       alpha = Ds*tf / X^2 = 4*Ds*tf / lambda2^2
%
%   with X = lambda2/2 the representative diffusion distance. Clyne and Kurz
%   replace the Brody-Flemings back-diffusion parameter beta by
%
%       beta = 2*alpha*(1 - exp(-1/alpha)) - exp(-1/(2*alpha))
%
%   which is bounded between 0 and 1. The liquid composition follows the
%   generalized back-diffusion form
```

```
%
%       Cl/c0 = [1 - (1 - beta*k)*fs]^((k-1)/(1 - beta*k))
%
%   and the interface solid obeys local equilibrium, Cs* = k*Cl. The average
%   solid composition follows from the solute balance:
%
%       CsAvg = [c0 - (1 - fs)*Cl] / fs
%
%   Limiting cases: beta -> 0 gives Scheil-Gulliver; beta -> 1 gives the
%   equilibrium lever rule.
%
%   Inputs
%       k         partition coefficient, k > 0 (not restricted to k <= 1)
%       c0        nominal composition
%       Ds        solid diffusivity (m^2/s)
%       tf        local solidification time (s)
%       lambda2   secondary dendrite arm spacing (m)
%       fs        solid fraction, 0 <= fs <= 1 (scalar or array)
%       alphaC    optional Won-Thomas coarsening constant (default 0)
%       useWT     optional flag for the Won-Thomas correction (default false)
%       betaOverride  optional direct value of beta in [0,1]. When supplied it
%                 replaces the alpha-derived beta, and alpha/useWT only affect
%                 the reported diagnostics.
%
%   Outputs (struct)
%       out.alpha      base Fourier number, 4*Ds*tf/lambda2^2
%       out.alphaEff   alpha used in beta (Won-Thomas corrected if requested)
%       out.beta       Clyne-Kurz back-diffusion parameter (direct value when
%                      betaOverride is supplied)
%       out.Cl         liquid composition
%       out.CsStar     interface solid composition, k*Cl
%       out.CsAvg      average solid composition
%       out.ClAvg      average liquid composition (= Cl; liquid is mixed)
%
%   Example (Al-4.8 wt.% Cu at 648 C)
%       out = clyne_kurz_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.9)
%       % alpha = 0.0048, beta = 0.0096
%       out = clyne_kurz_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.9, 0, false, 0.5)
%       % beta = 0.5 treated as an independent sensitivity variable
%
%   HOW TO RUN
%       addpath('models')                % from fs_slider_project
%       out = clyne_kurz_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.9);
%       out = clyne_kurz_model(0.173, 4.8, 4.8e-15, 100, 20e-6, ...
%                              linspace(0,0.99,200));     % vectorised in fs
%
%   AUTHOR
%       Prathamesh Deepak Ladhe, MSE 670 -- Solidification
%
%   SEE ALSO
%       won_thomas_model, scheil_model, equilibrium_model.
%       Derivation: report/TECHNICAL_REPORT.pdf, Section 2.5.

if nargin < 7 || isempty(alphaC)
    alphaC = 0;
end
if nargin < 8 || isempty(useWT)
    useWT = false;
end
if nargin < 9
    betaOverride = [];
end

```

```
lambda2 = max(lambda2, 1e-12);
alpha = 4*Ds*tf/(lambda2^2);

if useWT
    alphaEff = 2*(alpha + alphaC);
else
    alphaEff = alpha;
end

if ~isempty(betaOverride)
    beta = min(1, max(0, betaOverride));
    directBeta = true;
elseif alphaEff <= 0
    beta = 0;
    directBeta = false;
elseif isinf(alphaEff)
    beta = 1;
    directBeta = false;
else
    beta = 2*alphaEff*(1 - exp(-1/alphaEff)) - exp(-1/(2*alphaEff));
    directBeta = false;
end

if abs(k - 1) < 1e-12
    Cl = c0*ones(size(fs));
else
    q = beta*k - 1;
    if abs(q) < 1e-10
        Cl = c0*exp((1 - k)*fs);
    else
        Cl = c0*(1 + q*fs).^((1 - k)/q);
    end
end

CsStar = k*Cl;

CsAvg = zeros(size(fs));
small = fs < 1e-9;
if isscalar(fs)
    if small
        CsAvg = k*c0;
    elseif fs >= 1 - 1e-12
        CsAvg = c0;
    else
        CsAvg = (c0 - (1 - fs)*Cl)/fs;
    end
else
    CsAvg(small) = k*c0;
    idx = ~small & fs < 1 - 1e-12;
    CsAvg(idx) = (c0 - (1 - fs(idx)).*Cl(idx))./fs(idx);
    CsAvg(fs >= 1 - 1e-12) = c0;
end

out.alpha = alpha;
out.alphaEff = alphaEff;
out.beta = beta;
out.betaDirect = directBeta;
out.Cl = Cl;
out.CsStar = CsStar;
out.CsAvg = CsAvg;
out.ClAvg = Cl;
end
```

Listing 4: clyne_kurz_model.m

```m
function out = won_thomas_model(k, c0, Ds, tf, lambda2, fs, alphaC)
%WON_THOMAS_MODEL  Coarsening-corrected Clyne-Kurz microsegregation model.
%
%   out = won_thomas_model(k, c0, Ds, tf, lambda2, fs, alphaC)
%
%   Won and Thomas (2001) approximate the effect of dendrite coarsening by
%   replacing the Fourier number alpha in the Clyne-Kurz parameter with
%
%       alpha+ = 2*(alpha + alphaC),   alphaC = 0.1
%
%   The factor of two extends the result to columnar dendrites, and the extra
%   alphaC accounts for the smaller arm spacing present before coarsening is
%   complete. The concentration relation is unchanged:
%
%       Cl/c0 = [1 - (1 - beta*k)*fs]^((k-1)/(1 - beta*k))
%       beta  = 2*alpha+*(1 - exp(-1/alpha+)) - exp(-1/(2*alpha+))
%
%   This is an approximate correction, not a full coarsening model. The
%   empirical secondary-dendrite-arm-spacing fits in the Won-Thomas paper are
%   steel-specific and are not applied here.
%
%   Inputs
%       k        partition coefficient, k > 0 (not restricted to k <= 1)
%       c0       nominal composition
%       Ds       solid diffusivity (m^2/s)
%       tf       local solidification time (s)
%       lambda2  secondary dendrite arm spacing (m)
%       fs       solid fraction, 0 <= fs <= 1 (scalar or array)
%       alphaC   optional coarsening constant (default 0.1)
%
%   Outputs (struct)
%       out.alpha      base Fourier number, 4*Ds*tf/lambda2^2
%       out.alphaWT    corrected Fourier number, 2*(alpha + alphaC)
%       out.alphaEff   alias of alphaWT
%       out.beta       coarsening-corrected back-diffusion parameter
%       out.Cl         liquid composition
%       out.CsStar     interface solid composition, k*Cl
%       out.CsAvg      average solid composition
%       out.ClAvg      average liquid composition
%
%   Example (Al-4.8 wt.% Cu at 648 C)
%       out = won_thomas_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.9)
%       % alphaWT = 0.2096, beta = 0.3236
%
%   HOW TO RUN
%       addpath('models')                % from fs_slider_project
%       out = won_thomas_model(0.173, 4.8, 4.8e-15, 100, 20e-6, 0.9);
%
%   AUTHOR
%       Prathamesh Deepak Ladhe, MSE 670 -- Solidification
%
%   SEE ALSO
%       clyne_kurz_model, scheil_model, equilibrium_model.
%       Derivation: report/TECHNICAL_REPORT.pdf, Section 2.6.

if nargin < 7 || isempty(alphaC)
    alphaC = 0.1;
end

out = clyne_kurz_model(k, c0, Ds, tf, lambda2, fs, alphaC, true);
out.alphaWT = out.alphaEff;
end
```

Listing 5: won_thomas_model.m

C Glossary

Partition coefficient k Ratio of solid to liquid composition at the interface, C_S^*/C_L^*. For Al–Cu, $k \approx 0.173$.

Solid fraction f_s Fraction of the bar that has solidified; the slider's independent coordinate.

Back diffusion Diffusion in the already-formed solid that reduces the composition gradient left by segregation. Represented by α and β in Clyne–Kurz.

Boundary layer δ Characteristic liquid diffusion length ahead of the interface; $\delta/L = D_L/(vL)$.

Constitutional supercooling Liquid ahead of the interface that is below its local liquidus because solute enrichment outruns the imposed thermal gradient. Criterion in Eq. (29).

Lever rule Equilibrium solute balance between uniform solid and liquid.

Scheil–Gulliver No solid diffusion, perfectly mixed liquid.

Won–Thomas Coarsening correction that replaces α by $2(\alpha + \alpha_C)$.

D References and source material

1. MSE 670 handwritten notes, `Notes\_261030\_120238.pdf`, pages 1–9 (equilibrium, Scheil–Gulliver, boundary layer, liquidus profile and constitutional-supercooling criterion).
2. `solute\_redistribution\_detailed\_notes.pdf`, Chapters 5, 8, 9 (model derivations and the Won–Thomas discussion).
3. W. Kurz and D. J. Fisher, *Fundamentals of Solidification*, Trans Tech Publications.
4. T. F. Bower, H. D. Brody and M. C. Flemings, *Trans. AIME* **236** (1966) 624—634 (back-diffusion model).
5. T. W. Clyne and W. Kurz, *Metall. Trans. A* **12** (1981) 965—971 (back-diffusion parameter).
6. Y. M. Won and B. G. Thomas, *Metall. Mater. Trans. A* **32** (2001) 1755—1767 (coarsening correction).
7. M. C. Flemings, *Solidification Processing*, McGraw-Hill, 1974.

Report and code by Prathamesh Deepak Ladhe, MSE 670.
Built with base MATLAB, hand-written HTML/CSS/JavaScript and the equations documented above.

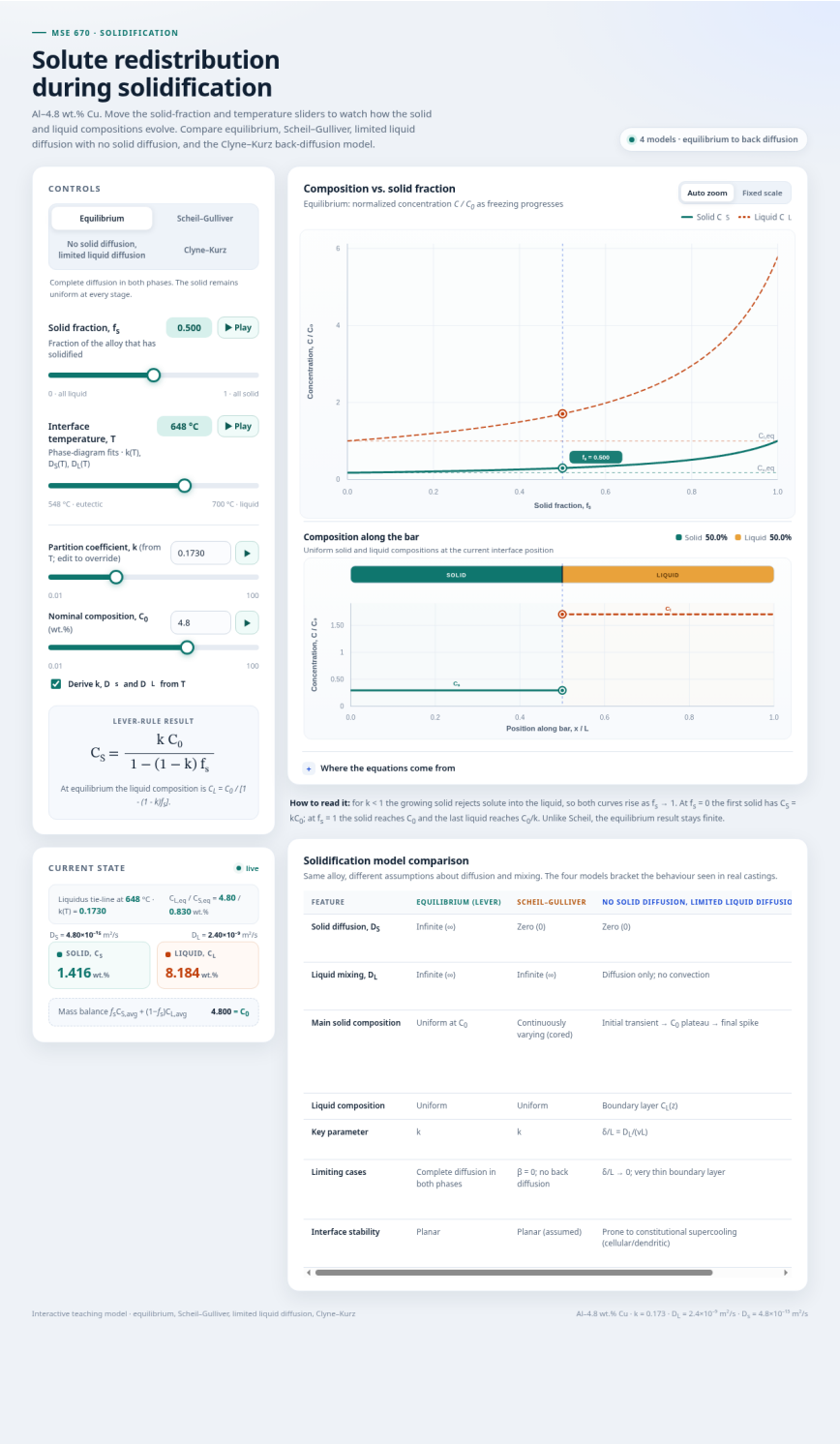


Figure 2: The browser app at the default state: controls and readout on the left, composition chart and along-bar chart on the right.

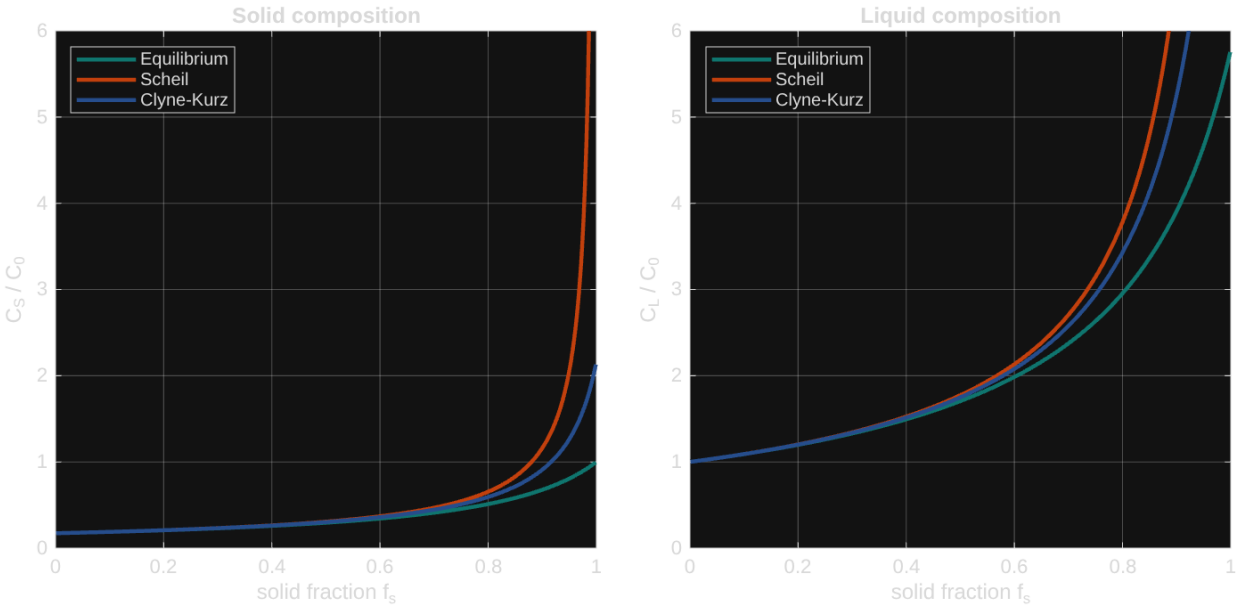


Figure 3: Normalised solid and liquid compositions for three of the models at the reference parameters ($k = 0.173$, $C_0 = 4.8\,\text{wt.\%}$). The limited-liquid model is spatial and is shown in Figure 4.

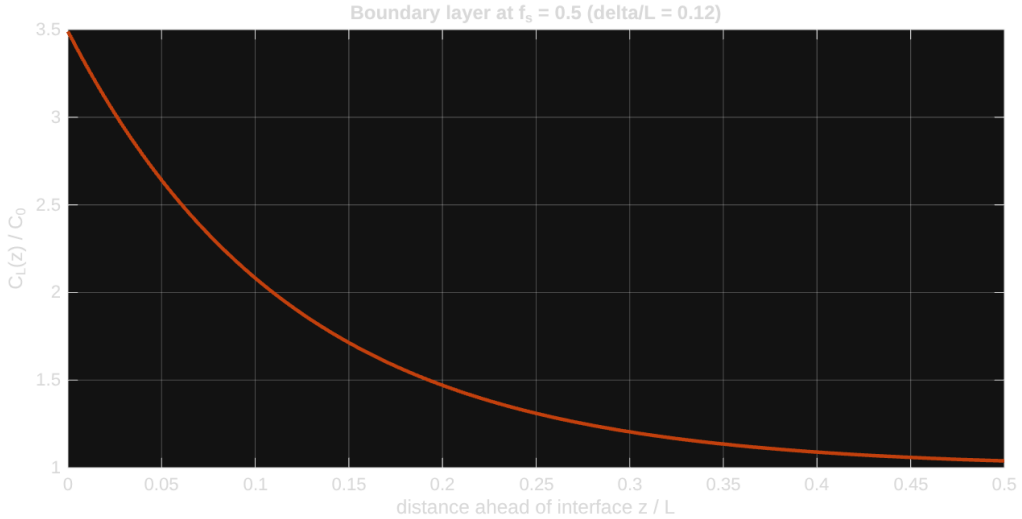


Figure 4: Liquid boundary layer ahead of the interface at $f_s = 0.5$ from Eq. (8), $\delta/L = 0.12$.